

Contenido

1. Introducción.	1
2. Instalación.	1
3. Conexión.	2
4. Esquemas.	2
5. Modelos.	3
6. Restricciones y validaciones.	3
7. Agregar documentos.	4
8. Búsqueda genérica con "buscar".	6
9. Búsqueda parametrizada con "find".	6
10. Eliminar documentos.	7
11. Actualizaciones de documentos.	8

1. Introducción.

Desde node podemos acceder a diferentes bases de datos pero es MongoDB la que veremos porque se ha vuelto muy popular en los últimos años, sobre todo para desarrollar con javascript. MongoDB en lugar de almacenar la información en tablas con sus correspondientes campos y registros, como hacen las bases de datos SQL, lo hace en estructuras de datos en formato BSON (similar a JSON), lo que facilita la integración con ciertas aplicaciones, como Node.

Muy a menudo, el acceso a mongoDB se desarrolla integrándolo con express, creando rutas de servidor para acceder a los diversos documentos, devolviendo datos en formato json al cliente que realiza la solicitud. En este tema estudiaremos cómo acceder a mongoDB pero dejaremos de lado express.

2. Instalación.

Es muy sencillo de instalar, se puede descargar desde:

<https://www.mongodb.com/try/download/community>

También contamos con la herramienta MongoDB Compass que nos ayuda a trabajar con bases de datos:

<https://www.mongodb.com/try/download/compass>

MongoDB está instalado como un servicio, debemos asegurarnos de que se está ejecutando antes de probar las aplicaciones que acceden a MongoDB.

3. Conexión.

Para conectarnos a las bases de datos de MongoDB, debemos usar una biblioteca llamada mongoose. Para probar esta biblioteca crearemos un proyecto de node que llamaremos test mongo, realizamos los pasos habituales:

- Crear una carpeta de prueba
- Dentro de esta carpeta, un archivo app.js vacío
- Ejecute npm init para generar package.json
- Agregue el script "start": "node ." en package.json
- Instalar mongoose: npm install mongoose

Ahora en app.js para conectarse a una base de datos llamada contactos de nuestro mongoDB:

```
// cargamos mongoose
const mongoose = require('mongoose');

// Configuramos promesas nativas
mongoose.Promise = global.Promise;

// conexión con la BD contacts
mongoose
  .connect('mongodb://localhost:27017/contacts')
  .then(() => {
    console.log('Conexión correcta a MongoDB');
  })
  .catch((err) => {
    console.error('Error en la conexión a la BD:', err);
  });
```

Si la base de datos de contactos no existe, como será el caso la primera vez que ejecutemos este proyecto, se creará (aunque no aparecerá hasta que le añadamos elementos en los pasos posteriores).

4. Esquemas.

Un esquema es un objeto en formato JSON que define como serán los documentos en una base de datos, para definir como serán nuestros contactos seguimos escribiendo en app.js:

```
//esquema
```

```
let contactSchema = new mongoose.Schema({
  Name: String,
  telephone: String,
  age: Number,
});
```

5. Modelos.

Una vez definido el esquema, necesitamos aplicarlo a un modelo para asociarlo así a una colección en la base de datos:

```
//modelo
let Contact = mongoose.model('contacts', contactSchema);
```

6. Restricciones y validaciones.

Si definimos un esquema simple como el ejemplo de contactos anterior, permitiremos agregar cualquier tipo de valor a los campos del documento. Así, por ejemplo, podríamos tener contactos sin nombre, o con edades negativas. Pero con Mongoose podemos proporcionar mecanismos de validación que permitan descartar automáticamente los documentos que no cumplan con las especificaciones:

- El validador **required** le permite definir que un determinado campo es obligatorio.
- El validador **default** le permite especificar un valor predeterminado para el campo, siempre que no se especifique ninguno.
- Los validadores **min** y **max** se utilizan para definir un rango de valores (mínimo y/o máximo) permitidos para datos de tipo numérico.
- Los validadores **minlength** y **maxlength** se utilizan para definir un tamaño mínimo o máximo de caracteres, en el caso de cadenas de texto.
- El validador **unique** indica que el campo en cuestión no admite duplicados (sería una clave alternativa, en un sistema relacional).
- El validador **match** se usa para especificar una expresión regular que debe coincidir, se puede ver en:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp

Haremos que el nombre y el número de teléfono sean obligatorios, y solo permitiremos edades entre 18 y 120 años (incluidas). Asimismo, el nombre tendrá una longitud mínima de 1 carácter, y el teléfono constará de 9 dígitos, utilizando una expresión regular, y será una clave única.

Podemos usar algún otro validador, como **trim**, para limpiar los espacios en blanco al principio y al final de los datos de texto. Con todas estas restricciones, el esquema se ve así:

```
//esquema
let contactSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    minlength: 1,
    trim: true,
  },
  telephone: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    match: /^\\d{9}$/,
  },
  age: {
    type: Number,
    min: 18,
    max: 120,
  },
});
```

Sustituimos el esquema anterior por este.

En las nuevas versiones de mongo, los índices únicos no se crean por defecto, por lo que tendremos que añadir estas líneas debajo de la creación del modelo si queremos que los campos unique funcionen:

```
// Creamos índices únicos
let pInit = Contact.init()
  .then(() => console.log('Índices creados'))
  .catch((err) => console.error(err));
```

7. Agregar documentos.

Para agregar contactos a nuestra base de datos:

```
// Añadir documentos
// primero creamos el contacto1
let contact1 = new Contact({
  name: 'Boris',
  telephone: '946112230',
  age: 49,
});
// usaremos una promesa save para guardarla en la BD
let p1 = contact1
  .save()
  .then((result) => {
    console.log('Contacto añadido:', result);
```

```

})
.catch((error) => {
  console.log('ERROR añadiendo contacto:', error);
});

// Como las promesas son asíncronas, usamos Promise.all para esperar a que se re-
suelvan.
// Solo cuando todas hayan terminado, cerramos la conexión con la base de datos.
Promise.all([p1,pInit]).then((values) => {
  mongoose.connection.close();
});

```

Si ahora ejecutamos el proyecto:

```

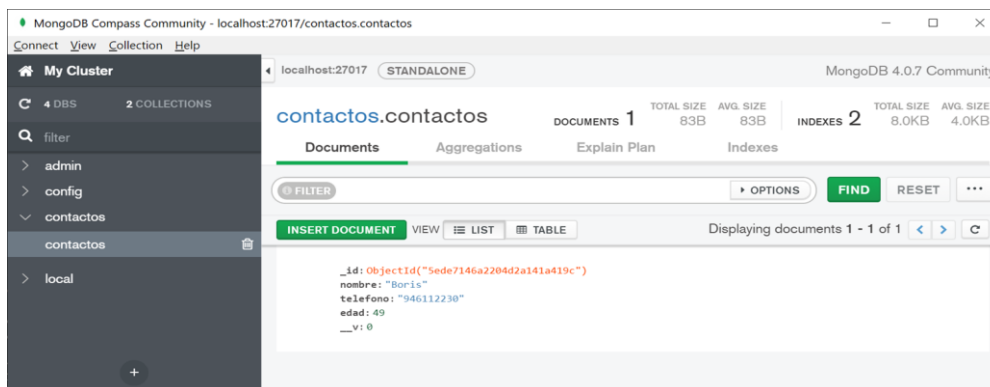
Conexión correcta a MongoDB
Contacto añadido: {
  name: 'Boris',
  telephone: '946112230',
  age: 49,
  _id: new ObjectId('68d278892ab54c330e2d7661'),
  __v: 0
}

```

Observe que obtenemos los mismos campos que definimos en el esquema (nombre, número de teléfono y edad) y dos campos adicionales que no hemos especificado:

- `__v` se refiere a la versión del documento. Inicialmente, todos los documentos parten de la versión 0 cuando se insertan, y luego esta versión se puede modificar cuando hacemos una actualización del documento.
- `_id` es un código generado automáticamente por Mongo, para cualquier documento en cualquier colección que tenga.

Si abrimos MongoDB Compass y nos conectamos a nuestro servidor mongoDB local, podemos ver que la base de datos de contactos, la colección de contactos y un contacto se han creado dentro de ella:



8. Búsqueda genérica con "buscar".

La forma más general de obtener documentos es utilizar el método de búsqueda asociado al modelo en cuestión. Podemos usarlo sin parámetros (con lo que obtendremos todos los documentos de la colección):

```
// buscamos con find
let p2 = Contact.find()
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log('ERROR:', error);
  });

// debemos esperar a que todas las promesas
// acaben para cerrar la conexión a la BD
Promise.all([p1, p2]).then((values) => {
  mongoose.connection.close();
});
```

Si ejecutamos, obtenemos esto:

```
Conexión correcta a MongoDB
ERROR añadiendo contacto: MongoServerError: E11000 duplicate key error collection: contacts.contacts index: telephone_1 dup key: { telephone: "946112230" }
    at InsertOneOperation.execute (C:\Users\beafu\Documents\BEA\SERPIS_25_26\BIN\2. nodejs\mongo\node_modules\mongodb\lib\operations\insert.js:51:19)
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5)
    at async tryOperation (C:\Users\beafu\Documents\BEA\SERPIS_25_26\BIN\2. nodejs\mongo\node_modules\mongodb\lib\operations\execute_operation.js:207:20)
    at async executeOperation (C:\Users\beafu\Documents\BEA\SERPIS_25_26\BIN\2. nodejs\mongo\node_modules\mongodb\lib\operations\execute_operation.js:75:16)
    at async Collection.insertOne (C:\Users\beafu\Documents\BEA\SERPIS_25_26\BIN\2. nodejs\mongo\node_modules\mongodb\lib\collection.js:157:16) {
  errorLabelSet: Set(0) {},
  errorResponse: {
    index: 0,
    code: 11000,
    errmsg: 'E11000 duplicate key error collection: contacts.contacts index: telephone_1 dup key: { telephone: "946112230" }',
    keyPattern: { telephone: 1 },
    keyValue: { telephone: '946112230' }
  },
  index: 0,
  code: 11000,
  keyPattern: { telephone: 1 },
  keyValue: { telephone: '946112230' }
}
[
  {
    _id: new ObjectId('68d27f2575308a88e0ee6b27'),
    name: 'Boris',
    telephone: '946112230',
    age: 49,
    __v: 0
  }
]
Indices creados
```

Devuelve un array con todos los contactos (aunque solo tenemos uno, devuelve un array) y también da error al intentar volver a insertar el mismo contacto ya que el teléfono tiene que ser único.

9. Búsqueda parametrizada con "find".

También podemos pasar un conjunto de criterios de búsqueda como parámetro para encontrar. Por ejemplo, para buscar contactos cuyo nombre sea "Boris" y su edad sea 49, haríamos esto:

```
// find con parámetros
let p3 = Contact.find({ name: 'Boris', age: 49 })
```

```

    .then((result) => {
      console.log(result);
    })
    .catch((error) => {
      console.log('ERROR:', error);
    });

// debemos esperar a que todas las promesas
// acaben para cerrar la conexión a la BD
Promise.all([pInit, p1, p2, p3]).then((values) => {
  mongoose.connection.close();
});

```

10. Eliminar documentos.

Para eliminar documentos de una colección, podemos utilizar el método **deleteOne** o **deleteMany**, que elimina los documentos que cumplen los criterios indicados como parámetro. Estos criterios se especifican de la misma manera que hemos visto para el método de búsqueda. Si no se especifican parámetros, se eliminan todos los documentos de la colección.

```

// borrar documentos
let p4 = Contact.deleteMany({ name: 'Boris' })
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log('ERROR:', error);
  });

// debemos esperar a que todas las promesas
// acaben para cerrar la conexión a la BD
Promise.all([pInit, p1, p2, p3, p4]).then((values) => {
  mongoose.connection.close();
});

```

El resultado obtenido en este caso contiene múltiples propiedades entre las que nos indican cuantos documentos se han borrado.

```
{ acknowledged: true, deletedCount: 1 }
```

Nota: es posible que las búsquedas anteriores devuelvan resultados vacíos, ya que todas las operaciones son asíncronas y el borrado del documento puede ejecutarse antes que las consultas. Para evitarlo, se deberían encadenar las promesas correctamente usando `.then`.

11. Actualizaciones de documentos.

Para realizar modificaciones en un documento de una colección, utilizaremos el método **findOneAndUpdate**. Comenta la promesa anterior p4 para evitar errores:

```
// Actualizar documentos
let p5 = Contact.findOneAndUpdate(
  { name: 'Boris' },
  { name: 'Boris Anaya', age: 50 },
  { new: true }
).then((result) => {
  console.log('Contact Updated:', result);
});

// debemos esperar a que todas las promesas
// acaben para cerrar la conexión a la BD
Promise.all([pInit, p1, p2, p3, p5]).then((values) => {
  mongoose.connection.close();
});
```

Lo que resulta en:

```
Contact Updated: {
  _id: new ObjectId('68d2858b82d4a42c5c024327'),
  name: 'Boris Anaya',
  telephone: '946112230',
  age: 50,
  age: 50,
  __v: 0
  age: 50,
  __v: 0
}
```

El problema de este método es que si hay varios documentos que cumplen la condición, solo se toma el primero. Cuando queremos asegurarnos de que actualizamos el documento que queremos, o buscamos un campo único, como un número de teléfono en nuestro caso, o usamos el método **findByIdAndUpdate**.